Studio Nx

Backend(FastAPI/Python) 83 — list_published()

2026-10-12

Backend(FastAPI/Python) 83 — `list_published()` (공개된 공지 목록)

항목	내용
파일	`studionx/backend/app/services/dev_note_service.py`
작업	P1-21 공지 API와 관리자 화면: 메인 페이지에 보일 공개 공지 목록 만들기 (PRD 01 FR-114)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	82 `DevNote` 모델 · 84 `get_published()` (다음)

1. 하는 일

공개해도 되는 공지를 **최근 공개순**으로 `limit`개 돌려줍니다. 메인 페이지의 Development 카드는 `limit=4`로 부릅니다. 보이는 조건은 함수 `published_filters()`에 모았습니다 (84번 상세도 같은 조건을 씁니다).

공지의 상태	목록에 보임?	이유
공개했고 공개 시각이 지남	보임	정상
초안 (`is_published = 0`)	안 보임	아직 공개 전
지운 공지 (`deleted_at`이 있음)	안 보임	소프트 삭제
공개 시각이 미래	안 보임	예약 공개, 그 시각이 되면 나타납니다
공개했는데 공개 시각이 비어 있음	안 보임	시각을 비교할 수 없어서 숨깁니다 (관리자 화면이 공개할 때 시각을 채웁니다)

정렬은 `published_at` 내림차순이고, 시각이 같으면 `note_id`가 큰 것(나중에 만든 것)이 먼저입니다.

2. 언제 쓰나

호출하는 곳	화면
85 `GET /dev-notes` 라우터 (다음 이후)	메인 페이지의 Development 카드 4개 (`home.js`의 `loadDevNotes`)

3. 코드

import

```
from datetime import UTC, datetime
from sqlalchemy import select
```

함수

```
def published_filters(now: datetime) -> list:
    return [
        DevNote.is_published.is_(True),
        DevNote.deleted_at.is_(None),
```

```
        DevNote.published_at <= now,
    ]


def list_published(db: Session, limit: int) -> list[DevNote]:
    now = datetime.now(UTC).replace(tzinfo=None)
    return list(
        db.scalars(
            select(DevNote)
            .where(*published_filters(now))
            .order_by(DevNote.published_at.desc(), DevNote.note_id.desc())
            .limit(limit)
        ).all()
    )
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻
`datetime.now(UTC).replace(tzinfo=None)`	지금의 UTC 시각을 시간대 표시 없이 만듭니다. DB에 저장된 값과 같은 기준입니다
`DevNote.is_published.is_(True)`	SQL의 `is_published IS TRUE`입니다
`DevNote.deleted_at.is_(None)`	SQL의 `deleted_at IS NULL`입니다
`DevNote.published_at <= now`	SQL의 `published_at <= '현재 시각'`입니다. 칸이 `NULL`이면 결과가 참이 아니라서 제외됩니다
`.where(*조건목록)`	목록의 조건을 모두 `AND`로 묶어 적용합니다
`.order_by(a.desc(), b.desc())`	`a` 내림차순, 같으면 `b` 내림차순입니다
`.limit(limit)`	최대 `limit`개만 가져옵니다
`db.scalars(...).all()`	한 줄에 모델 하나씩, 모두 가져옵니다

4. 흐름

1. 지금 시각(UTC)을 구합니다.
2. 공개되었고, 지워지지 않았고, 공개 시각이 지금 이전인 공지만 고릅니다.
3. 공개 시각이 최근인 순서로 정렬하고, `limit`개만 자릅니다.

5. 왜 이렇게

선택	이유
조건을 `published_filters()`로 따로 뺌	목록과 상세(84번)가 같은 기준이어야 "목록에는 없는데 주소를 치면 보이는" 일이 없습니다
예약 공개를 `published_at <= 지금`으로 처리	별도 스케줄러 없이도 시각이 되면 저절로 나타납니다
같은 시각은 `note_id` 내림차순	순서가 매번 같게 정해지고(안정적 정렬), 같은 시각에 올린 공지는 나중 것이 위에 옵니다
조건과 정렬이 `IDX(is_published, published_at)` 인덱스와 맞음	공지가 많아져도 빠르게 찾습니다

선택	이유
시각 비교를 DB(`NOW()`)가 아니라 파이썬의 UTC로	시험에서 시각을 바꿔 확인하기 쉽고, 저장 기준(UTC)과 같습니다
`limit`의 범위 검사는 라우터가 함	`limit`은 1~50이어야 하고, `Query(ge=1, le=50)`로 라우터에서 막습니다 (85번)

6. 시험 (`tests/test_dev_note_service.py`, 4개)

시험	결과
초안, 지운 것, 미래 시각, 시각 없음은 빠지고 정상 공지만 나옴	통과입니다
최신순이고, 같은 시각이면 `note_id`가 큰 것이 먼저	통과입니다
6개를 만들고 `limit` 4, 1	4개, 1개
30초 뒤 공개로 예약한 공지는 안 보이다가, 시각이 지나면 보임	통과입니다

4개 모두 통과했습니다. `ruff`도 통과했습니다.